Studio Nx

JS 311 — 템포 변경점을 재생, 재생선, 내보내기에 연결

2026-10-21

JavaScript 311 — 템포 변경점을 재생, 재생선, 내보내기에 연결 (`sequencer-devices.js`, `sequencer-audio.js`, `sequencer-arrange.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`, `studionx/frontend/static/js/pages/sequencer-audio.js`, `studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-13 템포 변경: `GET /full`의 템포 변경점을 담아 두고, 재생 세션, 재생선과 시간 표시, WAV 내보내기가 같은 목록으로 시간을 계산하게 함 (FR-202a)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 310 템포 API와 `/full`, 다음 없음 (S1-13 템포 변경의 마지막 문서)

1. 하는 일

오디오 쪽(`timing.js`, `scheduler.js`, `offline.js`)은 템포 변경을 처음부터 지원했습니다. `ticksToSecondsWithChanges`가 변경점 목록으로 시간을 계산하고, 내보내기의 길이와 범위 계산도 그 목록을 받습니다. 그런데 화면이 그 목록을 한 번도 넘기지 않았습니다. 재생 세션은 `getTempoChanges`를 받지 못했고, 내보내기 스냅샷은 `tempoChanges: []`를 고정으로 넘겼으며, 화면의 재생선과 시간 표시는 시작 템포 하나로 계산했습니다.

```
서버  GET /projects/{id}/full ─▶ tempo_changes: [ { tempo_change_id, tick, bpm, ramp } ]
화면  applyFullProject ─▶ indexTempoChanges: { tick, bpm, ramp } 로 정리해 담아 둠
        ├ playbackTempoChanges()를 PlaybackSession 의 getTempoChanges 로 넘김 ─▶ 재생 위치와 예약 시각이 템포 지도를 따름
        ├ exportSnapshot().tempoChanges 로 넘김 ─▶ 내보내기의 길이, 범위, 렌더링이 템포 지도를 따름
        └ setTempoChangesSource(playbackTempoChanges) ─▶ 재생선 이동과 상단 시간 표시가 템포 지도를 따름
```

고치기 전에는 템포 변경이 있는 곡이라도 재생과 내보내기, 재생선과 시간 표시가 모두 시작 템포 하나로 계산했습니다. 둘째 마디부터 템포가 절반인 곡은 내보낸 파일이 실제보다 짧았고 (3.87초, 올바른 길이는 5.81초), 재생선은 소리보다 앞서 달렸으며 시간 표시도 실제 경과 시간보다 작았습니다.

2. 언제 쓰나

쓰는 곳	하는 일
`applyFullProject` (273)	프로젝트를 열 때 `indexTempoChanges`로 변경점을 담습니다.
`getSession` (296)	재생 세션을 만들 때 `getTempoChanges: playbackTempoChanges`를 넘깁니다.
`exportSnapshot` (296)	내보내기 창이 쓰는 스냅샷의 `tempoChanges`에 `playbackTempoChanges()`를 넣습니다.
`initAudio` (296)	`setTempoChangesSource(playbackTempoChanges)`로 재생선과 시간 표시가 같은 목록을 읽게 합니다.
`play`의 재생선 이동, `updateTime` (`sequencer-arrange.js`)	변경점이 있으면 템포 지도로 위치와 시간을 계산합니다.

3. 코드

3.1 변경점 담기와 꺼내기 (`sequencer-devices.js`)

```
function indexTempoChanges(list) {
  tempoChanges = (list || []).map(function (c) {
    return { tick: Number(c.tick), bpm: Number(c.bpm), ramp: c.ramp === 'LINEAR' ? 'LINEAR' : 'STEP' };
  }).filter(function (c) {
    return isFinite(c.tick) && c.tick > 0 && isFinite(c.bpm) && c.bpm > 0;
  }).sort(function (a, b) { return a.tick - b.tick; });
}
```

```
export function playbackTempoChanges() {
  return tempoChanges.map(function (c) { return { tick: c.tick, bpm: c.bpm, ramp: c.ramp }; });
}
```

3.2 오디오와 연결 (`sequencer-audio.js`)

바뀐 곳은 세 군데이고, 모두 한 줄입니다. 위의 함수를 `import`에 더하고 아래처럼 넘깁니다.

```
// getSession 안: 재생 세션이 템포 지도를 받음 (이전에는 이 줄이 없었음)
session = new PlaybackSession({
  getTracks: playbackTracks,
  ...
  getBpm: currentBpm,
  getTempoChanges: playbackTempoChanges,
  ...
});

// exportSnapshot 안: 이전에는 tempoChanges: [] 로 고정
tempoChanges: playbackTempoChanges(),

// initAudio 안: 재생선과 시간 표시가 같은 템포 지도를 읽음
setTempoChangesSource(playbackTempoChanges);
```

3.3 재생선과 시간 표시 (`sequencer-arrange.js`)

재생선은 소리 엔진과 따로 움직입니다. 화면이 `requestAnimationFrame`마다 지난 시간만큼 위치를 옮기는데, 이전에는 "시작 템포에서 1초에 몇 마디"를 곱했습니다. 템포 변경점이 있으면 지금 위치의 시각(초)에 지난 시간을 더한 뒤 그 시각의 위치(tick)로 바꿉니다. 시간 표시도 같은 변환을 씁니다.

```
export function setTempoChangesSource(fn) {
  tempoSource = typeof fn === 'function' ? fn : null;
}
```

```
function currentTempoChanges() {
  return tempoSource ? tempoSource() : [];
}
```

```
function play() {
  const c = playheadCtx;
  c.playing = true;
```

```
  c.startBar = c.bar;
  c.anchor = null;
  if (transportHooks.onPlay) transportHooks.onPlay(c.bar);
  document.body.classList.add('is-playing');
  let last = performance.now();
  const tick = (now) => {
    if (!c.playing) return;
    const dt = (now - last) / 1000;
    const changes = currentTempoChanges();
    let next;
    if (changes.length) {
      // 템포 변경점이 있으면 지금 위치의 시각에 지난 시간을 더한 뒤 다시 위치(tick)로 바꿈 (소리와 같은 템
포 지도)
      const sec = ticksToSecondsWithChanges(c.bar * TICKS_PER_BAR, currentBpm(), changes);
      next = secondsToTicks(sec + dt, currentBpm(), changes) / TICKS_PER_BAR;
    } else {
      next = c.bar + dt * (currentBpm() / 60 / 4);     // 4/4, 템포가 하나일 때
    }
    // Loop 켜짐: 구간 끝을 지나면 구간 처음으로 (구간보다 뒤에서 시작했으면 그대로 진행)
    const loopEnd = loopState.start + loopState.length;
    if (loopState.on && loopState.length > 0 && c.bar < loopEnd && next >= loopEnd) next = loopState.sta
rt + ((next - loopEnd) % loopState.length);
    last = now;
    if (next >= totalBars(c.lanes)) { stop(); return; }
    setPlayheadBar(next);
    c.raf = requestAnimationFrame(tick);
  };
  c.raf = requestAnimationFrame(tick);
}
```

```
function updateTime(bar) {
  const main = document.getElementById('time-main');
  const small = document.getElementById('time-bars');
  if (!main || !small) return;
  const changes = currentTempoChanges();
  const sec = changes.length
    ? ticksToSecondsWithChanges(bar * TICKS_PER_BAR, currentBpm(), changes)
    : (bar * 4 * 60) / currentBpm();
  const mm = String(Math.floor(sec / 60)).padStart(2, '0');
  const ss = String(Math.floor(sec % 60)).padStart(2, '0');
  const ms = String(Math.floor((sec * 1000) % 1000)).padStart(3, '0');
  const clock = `${mm}:${ss}:${ms}`;
  const bars = `${Math.floor(bar) + 1}.${Math.floor((bar % 1) * 4) + 1}.${Math.floor(((bar * 4) % 1) *
4) + 1}`;
  const barsFirst = document.getElementById('time')?.dataset.mode === 'bars';
  main.textContent = barsFirst ? bars : clock;
  small.textContent = barsFirst ? clock : bars;
}
```

4. 최신 문법으로 쓰면

```
// indexTempoChanges 를 최신 문법으로 쓴 모양
const indexTempoChanges = (list = []) => {
  tempoChanges = list
```

```
    .map(({ tick, bpm, ramp }) => ({ tick: Number(tick), bpm: Number(bpm), ramp: ramp === 'LINEAR' ? 'LI
NEAR' : 'STEP' }))
    .filter(({ tick, bpm }) => Number.isFinite(tick) && tick > 0 && Number.isFinite(bpm) && bpm > 0)
    .sort((a, b) => a.tick - b.tick);
};

export const playbackTempoChanges = () => tempoChanges.map((c) => ({ ...c }));
```

전통 문법 (실제 코드)	최신 문법
`(list || [])`	기본 매개변수 `list = []`
`function (c) { return { ... }; }`	화살표 함수
`{ tick: c.tick, bpm: c.bpm, ramp: c.ramp }`로 복사	객체 전개 `{ ...c }`
`isFinite(x)`	`Number.isFinite(x)`

5. 왜 이렇게

선택	이유
목록을 `sequencer-devices.js`에 둡니다	클립, 패턴, 트랙처럼 `GET /full`로 받은 프로젝트 데이터를 이 파일이 이미 담고 있고, 재생과 내보내기가 `playbackTracks`, `playbackInstruments`와 같은 방식으로 읽어 갑니다. 새 저장소를 만들 필요가 없습니다.
재생과 내보내기가 같은 함수를 부릅니다	들리는 소리와 파일의 소리가 같아야 한다는 원칙(S1-11b)을 템포에도 적용합니다. 한쪽만 고치면 두 결과가 어긋납니다.
`playbackTempoChanges`는 복사본을 돌려줍니다	엔진이 목록을 정렬하거나 고쳐도 화면이 가진 원본이 변하지 않게 합니다.
숫자가 아니거나 `tick`이 0 이하인 줄은 버립니다	`timing.js`는 이런 줄을 만나면 오류를 던져 재생이 멈춥니다. 서버가 이미 검증하지만, 화면 쪽에서도 한 번 거르면 이상한 데이터 하나가 재생 전체를 막지 않습니다.
`ramp`는 `LINEAR`만 인정하고 나머지는 `STEP`입니다	엔진의 규칙과 같습니다 (`timing.js`가 `LINEAR`가 아니면 `STEP`으로 취급).
재생선은 위치를 시각으로 바꿨다가 다시 위치로 바꿉니다	템포가 바뀌는 곳을 지나는 프레임에서도 정확하게 이어지게 하려는 것입니다. 현재 템포를 곱하는 방식은 변경점을 지나는 프레임에서 오차가 생깁니다.
변경점이 없으면 이전 계산 그대로입니다	템포가 하나뿐인 곡(대부분의 곡)은 바뀐 것이 없고, 기존 시험이 모두 그대로 통과합니다.
재생선 등록은 함수 하나로 합니다 (`setTempoChangesSource`)	`sequencer-arrange.js`가 `sequencer-devices.js`를 `import`하면 서로를 부르는 순환이 생깁니다 (devices가 이미 arrange를 `import`). 이미 있는 `setTransportHooks`와 같은 방식으로 오디오 연결 파일이 목록을 알려 주게 했습니다.

6. 알려진 한계

한계	내용
템포를 화면에서 편집하는 방법이 아직 없습니다	템포 줄(Tempo lane)은 `sequencer.js`의 `renderGlobalLanes`로 정해져 있지만 아직 구현되지 않은 템플릿입니다. 지금은 API(`PUT /projects/{id}/tempo-changes`)로만 변경점을 만들 수 있습니다.
박자 변경은 아직 반영하지 않습니다	재생선과 시간 표시는 4/4로 계산합니다. 박자 변경점(`meter_changes`)은 모델과 API가 아직 없습니다.

한계	내용
마디 칸의 눈금은 그대로입니다	마디선 간격은 마디 단위라 템포와 무관해서 바뀌지 않습니다. 템포가 바뀌면 같은 마디가 더 오래 또는 짧게 걸릴 뿐입니다.
딜레이의 템포 동기(Sync)는 시작 템포를 씁니다	이펙트가 받는 `getBpm`은 곡 전체에서 하나의 값입니다. 곡 중간에서 템포가 바뀌어도 Sync 딜레이의 간격은 바뀌지 않습니다.

7. 시험

`docs/handoff/s1-13_tests/ui_tempo.mjs`(20건)가 실제 화면(헤드리스 Chrome)에서 확인합니다. 시험 전용 오디오 트랙에 8초 사인파 클립(2마디)을 놓고, 둘째 마디부터 템포를 절반으로 낮춥니다.

- 서버: `GET /full`의 `tempo_changes`에 변경점 한 건이 있고 `bpm`이 숫자입니다.
- 화면: `playbackTempoChanges()`와 `exportSnapshot().tempoChanges`가 같은 목록(tick 3840, 절반 템포, STEP)이고, 상단 BPM은 프로젝트의 시작 템포 그대로입니다.
- 재생: 엔진의 `transport.changes`가 변경점을 받았고, 재생 위치가 템포 지도로 계산한 위치와 200 tick 안으로 맞으며 시작 템포 그대로라면 있을 위치보다 500 tick 넘게 뒤에 있습니다.
- 재생선과 시간 표시: 재생선이 엔진 위치와 0.2마디 안으로 맞고(화면은 곧바로 시작하고 엔진은 샘플을 불러온 뒤 시작하므로 그 차이를 허용), 시작 템포로 계산했을 위치보다 분명히 뒤에 있습니다. 상단 시간 표시가 소리의 경과 시간과 0.6초 안으로 맞고, 시작 템포로 계산한 시간과는 0.5초 넘게 다릅니다.
- 내보내기 창: 보여 주는 전체 길이가 템포 지도 기준(5.8초 근처)이고, 변경을 무시한 길이(3.9초)와 다릅니다. 실제로 내려받은 WAV의 길이가 5.81초이고, 느려진 둘째 마디의 끝까지 소리가 이어집니다.
- 되돌려 보기: `sequencer-audio.js`의 연결만 원래대로 돌려놓고 시험을 돌리면(재생선과 시간 표시를 넣기 전의 16건 시험) 6건이 실패합니다 (엔진이 변경점을 못 받고, 파일 길이 3.87초). 즉 이 시험이 이 오류를 실제로 잡습니다.
- 시험 중 발견한 문제는 없었습니다. 위 되돌려 보기가 이 작업에서 고친 오류(재생과 내보내기가 템포 변경을 무시)의 증거입니다.